

Security Audit

BiohackerDAO (DAO)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	23
Conclusion	24
Our Methodology	25
Disclaimers	27
About Hashlock	28

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The BioHackerDAO team partnered with Hashlock to conduct a security audit of their DataLiquidityPoolImplementation.sol smart contract. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

BiohackerDAO is a pioneering decentralized autonomous organization (DAO) that unites a diverse community—including founders, scientists, developers, crypto enthusiasts, influencers, and venture capitalists—to revolutionize human enhancement through data-driven self-optimization. By conducting decentralized self-enhancement experiments, BiohackerDAO responsibly collects, analyzes, and monetizes the resulting data, aiming to build the most powerful hive mind in the field of data-driven self-optimization.

With over 100,000 data points generated from various experiments, the organization is committed to gradually pursuing radical human enhancement and seeding superhumans. Operating as a DAO, BiohackerDAO principles of Web3 by fostering a collaborative, transparent, and decentralized approach to advancing human potential.

Project Name: BioHackerDAO

Project Type: DLP

Compiler Version: 0.8.24

Website: www.biohackerdao.org

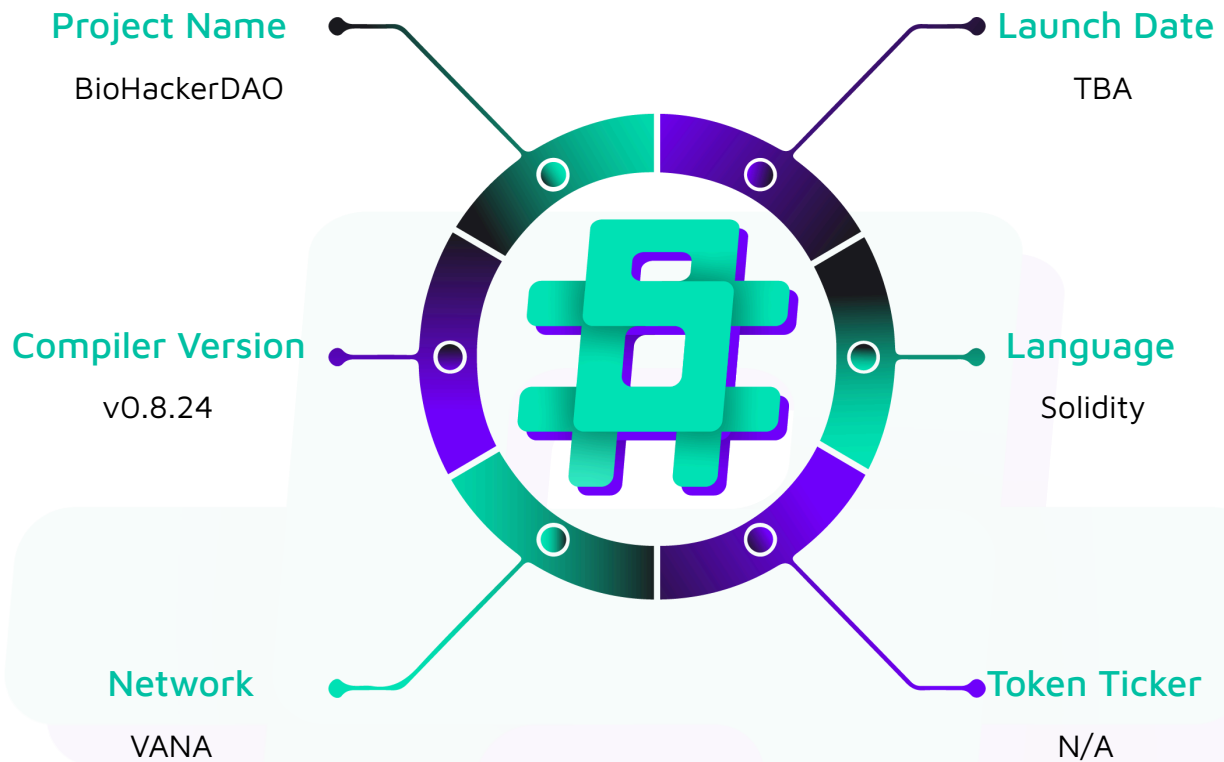
Logo:



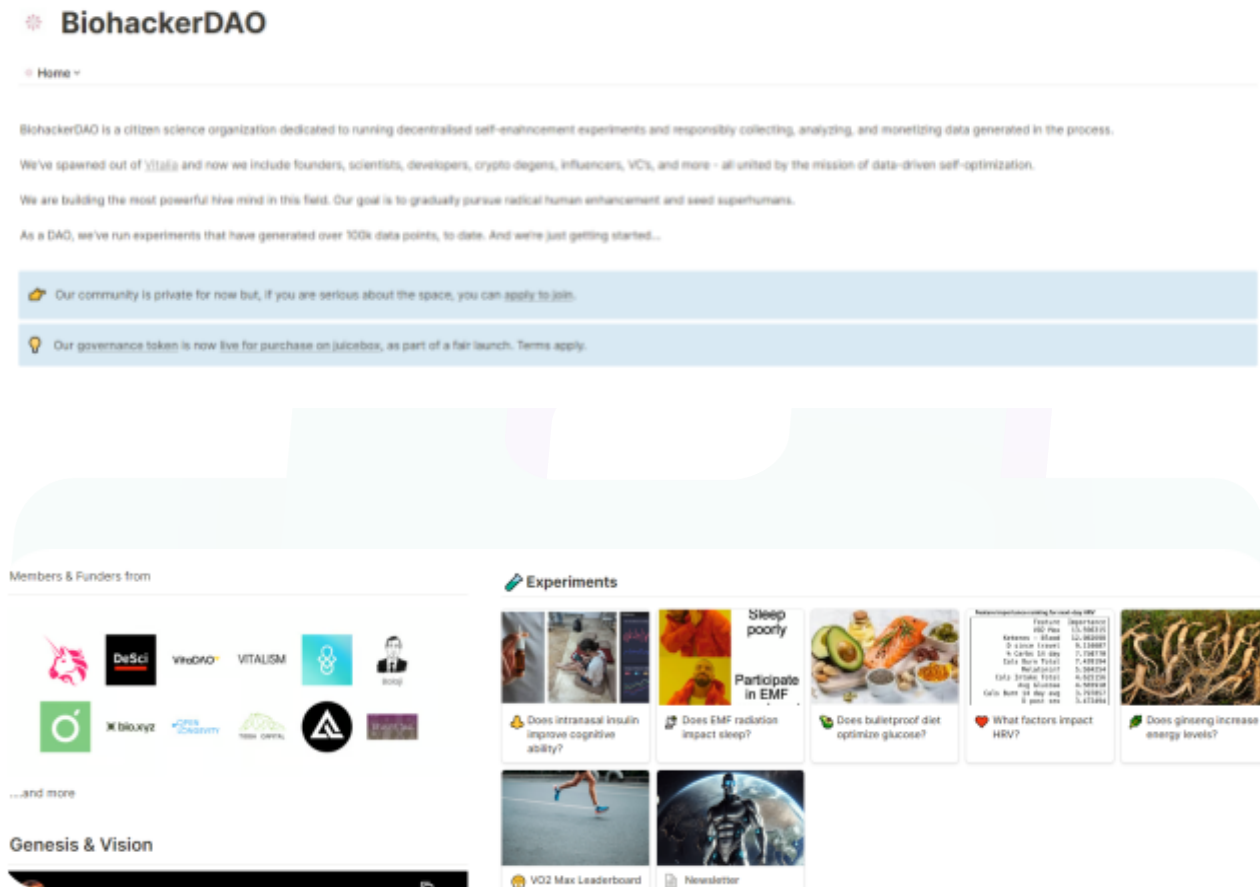
BiohackerDAO



Hashlock Pty Ltd

Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the BioHackerDAO project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	BioHackerDAO Protocol Smart Contracts
Platform	Vana / Solidity
Audit Date	December, 2024
Contract 1	DataLiquidityPoolImplementation.sol
Contract 1 MD5 Hash	a228963ea83674c4036866509db398f6
GitHub Commit Hash	2bf530d0eb49cfa5ea90cfca50a0e1fe71a24777

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerability

5 Low severity vulnerabilities

1 Gas Optimisation

2 QAs

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
DataLiquidityPoolImplementation.sol - Fork of Vana DLP contract without the ERC20 interactions. - Allows users to: - Request rewards for submitted files with valid proofs - Allows admins to: - Update protocol parameters (reward factors, public key, proof instructions) - Manage integrations with other contracts (DataRegistry, TeePool) - Pause/unpause the contract	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the BioHackerDAO project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the BioHackerDAO project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] DataLiquidityPoolImplementation#requestReward - The same `fileId` can be used repeatedly which will cause manipulation of state variables

Description

The `requestReward` function allows users to receive rewards for the files they have presented into the protocol. However, this function allows the same file to be processed multiple times as the checks are insufficient, leading to inflated contributor counts, corrupted contributor tracking in the contract's state and faulty event emissions.

Vulnerability Details

The `requestReward` function allows users to receive rewards for the files they have presented into the protocol.

```
function requestReward(uint256 fileId, uint256 proofIndex) external override
whenNotPaused nonReentrant {
    IDataRegistry.Proof memory fileProof = dataRegistry.fileProofs(fileId,
proofIndex);

    if (keccak256(bytes(fileProof.data.instruction)) !=
keccak256(bytes(proofInstruction))) {
        revert InvalidProof();
    }

    File storage file = _files[fileId];

    if (file.rewardAmount != 0) {
        revert FileAlreadyAdded();
    }
}
```

```

IDataRegistry.FileResponse memory registryFile = dataRegistry.files(fileId);

if (fileProof.data.score > 1e18 || fileProof.data.score == 0) {
    revert InvalidScore();
}

bytes32 _messageHash = keccak256(
    abi.encodePacked(
        registryFile.url,
        fileProof.data.score,
        fileProof.data.dlpId,
        fileProof.data.metadata,
        fileProof.data.proofUrl,
        fileProof.data.instruction
    )
);

address signer =
_messageHash.toEthSignedMessageHash().recover(fileProof.signature);

if (!teePool.isTee(signer)) {
    revert InvalidAttestator();
}

file.timestamp = block.timestamp;
file.proofIndex = proofIndex;
file.rewardAmount = 0;

_filesList.add(fileId);

Contributor storage contributor = _contributorInfo[registryFile.ownerAddress];
contributor.filesList.add(fileId);

if (contributor.filesList.length() == 1) {

```

```

        contributorsCount++;
        _contributors[contributorsCount] = registryFile.ownerAddress;
    }

    emit RewardRequested(registryFile.ownerAddress, fileId, proofIndex,
file.rewardAmount);
}

```

The code snippet highlighted in yellow, prevents the same file from being added as every file is supposed to have a reward. However, observed in the code snippet highlighted in green, after file rewards are set to 0 before being added. This means that the implemented check is insufficient and the same file can be used in the function call more than once.

The subsequent `.add` call will fail silently as it will not revert on failure and return false. Meaning that the same file will not be added but the function will not revert.

The following if check highlighted in red, will pass as the file has been added only once and it will cause the same user to be added as a contributor and increase the contributors count.

The function will then incorrectly emit the `RewardRequested` event even though no files have been added.

Impact

Manipulation of relevant state variables and misleading event emissions. This is increasingly more problematic when these state variables are used in off-chain and future on-chain systems.

Recommendation

Implement checks in the function to check for the returned value of `add()` calls. If this call returns false, revert.

Alternatively, set the reward amount to be greater than 0.

Status

Resolved

Low

[L-01] DataLiquidityPoolImplementation#initialize - Missing checks for `address(0)` in initializer

Description

It is good practice to implement checks for `address(0)` to prevent accidental inputs that can lead to the need for redeploying contracts. Instances are shown below.

```
function initialize(InitParams memory params) external initializer {  
    // rest of the function  
    dataRegistry = IDataRegistry(params.dataRegistryAddress);  
    teePool = ITeePool(params.teePoolAddress);  
    . . .  
    _transferOwnership(params.ownerAddress);  
}
```

Recommendation

It is strongly recommended to implement checks to prevent the zero address from being set during the initialization of contracts. This can be achieved by adding required statements that ensure address parameters are not the zero address.

Status

Acknowledged

[L-02] DataLiquidityPoolImplementation#updateTeePool, - Missing checks for `address(0)` when updating state variables

Description

It is good practice to implement checks for `address(0)` to prevent accidental inputs that can lead to unexpected functionality.

Recommendation

It is strongly recommended to implement checks to prevent the zero address from being set with these functions. This can be achieved by adding required statements that ensure address parameters are not the zero address. An example is shown below.

```
function updateTeePool(address newTeePool) external override onlyOwner {
    require(newTeePool != address(0), "Invalid tee pool");

    teePool = ITeePool(newTeePool);

    emit TeePoolUpdated(newTeePool);
}
```

Status

Acknowledged

[L-03] DataLiquidityPoolImplementation#addRewardsForContributors - Lack of access control allows anyone to increase the state variable with any value

Description

The `addRewardsForContributors` function does not have any access control and does not achieve anything other than incrementing the state variable with the given input. This allows anyone to increase this unused state variable to any value they want.

```
function addRewardsForContributors(uint256 contributorsRewardAmount) external override
nonReentrant {

    totalContributorsRewardAmount += contributorsRewardAmount;

}
```

Recommendation

Implement access control or remove the function/function's logic.

Status

Resolved

[L-04] DataLiquidityPoolImplementation - Upgradeable contract is missing `__gap[...]` storage variable

Description

Storage gaps are needed to not break storage layouts when adding new variables to base contracts. Listed contracts may not be inherited but it is good practice to add them now rather than forgetting later.

Recommendation

When designing upgradeable contracts, it's important to include `__gap[...]` storage variables as storage gaps to prevent issues with storage layout changes when adding new variables to base contracts in the future. While the listed contracts may not be directly inherited from, it is a good practice to add `__gap[...]` storage variables now to ensure a robust and future-proof upgradeable contract design. This proactive approach helps avoid potential complications and storage layout conflicts in the later stages of contract development.

Status

Acknowledged

[L-05] DataLiquidityPoolImplementation - Centralization risk for privileged roles

Description

The project uses the `Ownable2StepUpgradeable` contract from `OpenZeppelin` to implement access control for critical functions. This introduces a single point of failure as malicious/accidental updates can cause serious issues in contract's functionality.

```
function pause() external override onlyOwner {
function unpaue() external override onlyOwner {
function updateFileRewardFactor(uint256 newFileRewardFactor) external override onlyOwner {
{
function updateTeePool(address newTeePool) external override onlyOwner {
```

```
function updateProofInstruction(string calldata newProofInstruction) external override  
onlyOwner {  
  
    function updatePublicKey(string calldata newPublicKey) external override onlyOwner {
```

Recommendation

To mitigate centralization risks associated with privileged functions, consider implementing a multi-signature or decentralized governance mechanism. Instead of relying solely on a single owner/administrator, distribute control and decision-making authority among multiple parties or stakeholders. This approach enhances security, reduces the risk of malicious actions by a single entity, and prevents single points of failure.

Status

Acknowledged

Gas

[G-01] DataLiquidityPoolImplementation - Prevent setting a state variable with the same value

Description

Not only is it wasteful in terms of gas, but this is especially problematic when an event is emitted and the old and new values set are the same, as listeners might not expect this kind of scenario. Instances are shown below

```
function updateFileRewardFactor(uint256 newFileRewardFactor) external override onlyOwner {
    function updateTeePool(address newTeePool) external override onlyOwner {
        function updateProofInstruction(string calldata newProofInstruction) external override onlyOwner {
            function updatePublicKey(string calldata newPublicKey) external override onlyOwner {
```

Recommendation

Implement checks in your functions to avoid setting state variables to their existing values. Prior to updating a state variable, compare the new value with the current value and proceed with the assignment only if they differ. An example is shown below.

```
function updateFileRewardFactor(uint256 newFileRewardFactor) external override onlyOwner {
    if (_fileRewardFactor == newFileRewardFactor) {
        revert FileRewardFactorNotChanged();
    }
    _fileRewardFactor = newFileRewardFactor;
    emit FileRewardFactorUpdated(newFileRewardFactor);
}
```

Status

Acknowledged

QA

[Q-01] DataLiquidityPoolImplementation#requestReward - The `nonReentrant` modifier should occur before all other modifiers

Description

In Solidity, the order in which modifiers are applied to a function can significantly impact the function's behavior and security. The `nonReentrant` modifier is crucial for preventing reentrancy attacks, a common vulnerability in smart contracts. This modifier should be the first in the list of applied modifiers to ensure that the reentrancy check is performed before any other logic in the function. Placing `nonReentrant` after other modifiers could potentially lead to security weaknesses, as other modifiers might perform state changes or external calls before the reentrancy protection takes effect. Therefore, correct positioning of the `nonReentrant` modifier is essential for maintaining the integrity and security of the function and, by extension, the entire contract.

Recommendation

To ensure the effectiveness of the `nonReentrant` modifier in protecting against reentrancy, it should be placed as the first modifier in a function's list of modifiers, before all other modifiers. This helps prevent reentrancy attacks from being triggered by other modifiers.

Status

Acknowledged

[Q-02] DataLiquidityPoolImplementation - Constants in comparisons should appear on the left side

Description

Doing so will prevent typo bugs.

Recommendation

To prevent typo bugs and improve code readability, it's advisable to place constants on the left side of comparisons. This coding practice helps catch accidental assignment (=) instead of comparison (==) and enhances code quality.

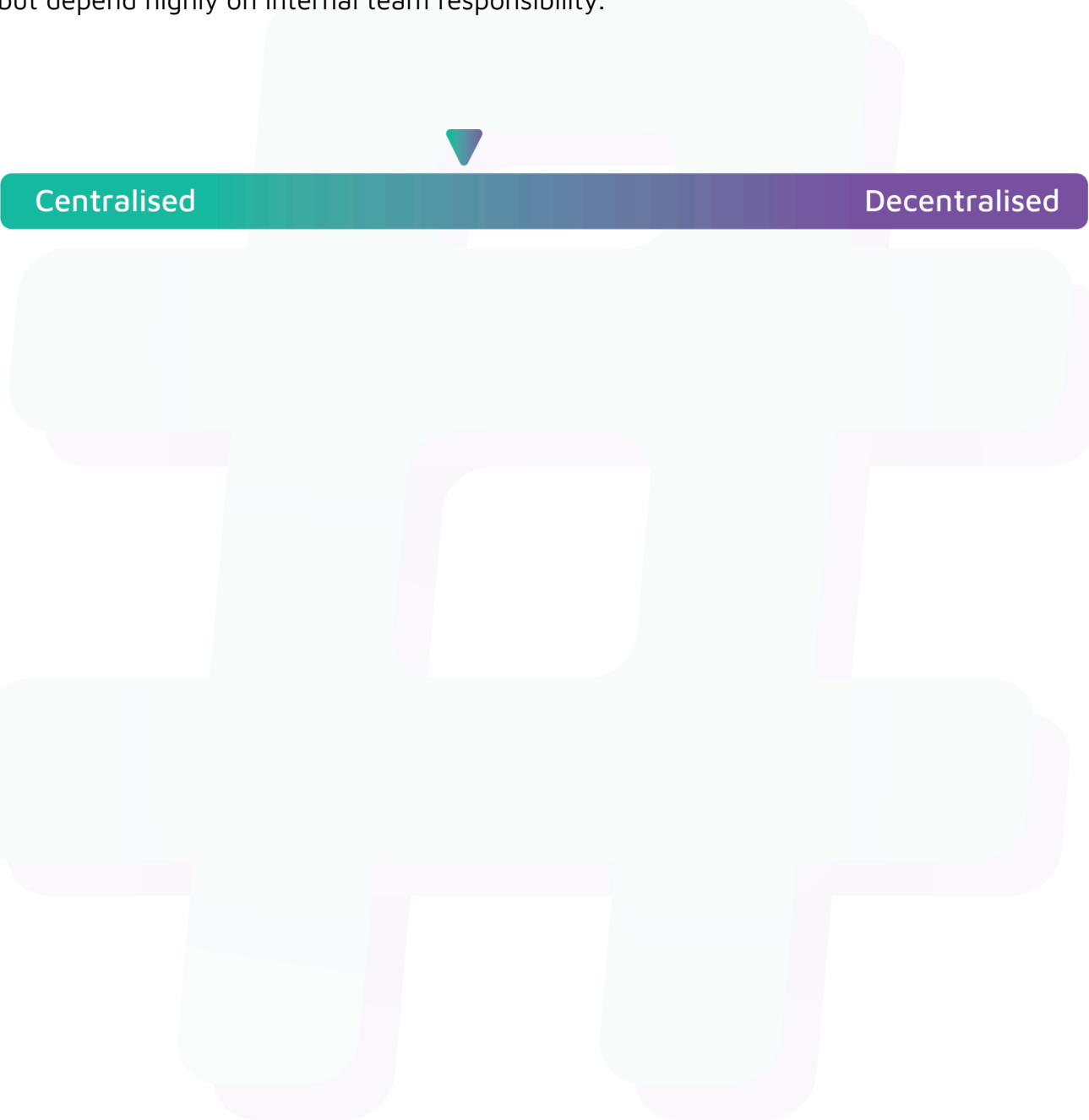
Status

Acknowledged

Centralisation

The BioHackerDAO project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the BioHackerDAO project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.